
Affichage Avancé et Menu Interactif

LCD, OLED, Matrice LED 8x8 et anneau lumineux piloté par boutons

Darix SAMANI SIEWE
Rapport de projet – Systeme a Microcontrolleurs

1^{er} juillet 2026

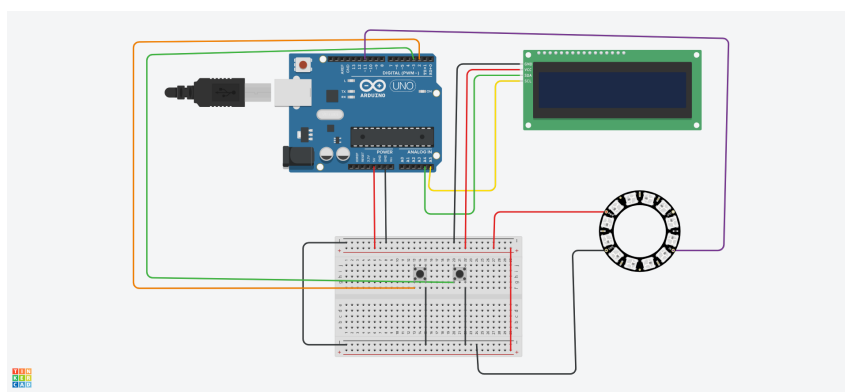


FIGURE 1 – Vue du montage du projet (capture Tinkercad ou photo réelle à insérer ici).

Table des matières

1	Introduction	2
2	Description du Projet	2
3	Composants Utilisés	2
3.1	Composants du cahier des charges (version complète)	3
3.2	Composants utilisés dans la simulation Tinkercad	3
4	Notions Électroniques Clés	3
5	Schéma du Montage	4
6	Code Arduino	4
7	Analyse et Discussion du Code	6
8	Ressources Complémentaires	7
9	Conclusion	7

1 Introduction

Ce rapport présente un projet dédié à la **maîtrise des interfaces d’affichage** avec la carte **Arduino Uno**. Le cahier des charges initial prévoit l’exploitation de plusieurs technologies d’affichage complémentaires : texte défilant sur un écran **LCD 16x2**, graphiques et icônes sur un écran **OLED SSD1306**, animations personnalisées sur une **matrice LED 8x8 MAX7219**, et un compteur sur un afficheur **7 segments TM1637**. L’ensemble est piloté par un **système de menus interactif** commandé via des boutons poussoirs.

Dans la version simulée sous **Tinkercad** présentée ici, le système de menus a été implémenté avec un **écran LCD I²C** et un **anneau lumineux NeoPixel (12 LED RGB adressables)** piloté par deux boutons, ce qui permet de valider concrètement la logique de navigation de menu, l’anti-rebond et le déclenchement d’animations lumineuses, en attendant l’intégration des autres modules d’affichage (OLED, matrice MAX7219, TM1637).

2 Description du Projet

Le projet repose sur un **menu déroulant** affiché sur l’écran LCD, que l’utilisateur parcourt à l’aide de boutons poussoirs. Dans la version complète, ce menu doit permettre de sélectionner différents modes d’affichage ou d’animation répartis sur les quatre périphériques visés (LCD, OLED, matrice LED, afficheur 7 segments). Dans la version simulée, le menu propose trois actions appliquées à un **anneau de LED NeoPixel** :

- **Chenillard** : une LED bleue parcourt l’anneau, une position à la fois ;
- **Couleurs** : affichage statique d’un dégradé multicolore réparti sur les 12 LED ;
- **Effacer** : extinction complète de l’anneau.

La navigation se fait via deux boutons : un bouton « **Suivant** » qui fait défiler les options du menu (affichées avec un curseur > sur le LCD), et un bouton « **Valider** » qui déclenche l’action sélectionnée. Cette structure de menu, bien que simplifiée par rapport au projet final, constitue le squelette logiciel directement réutilisable pour piloter les autres écrans (OLED, matrice MAX7219, TM1637) une fois ceux-ci intégrés.

Objectif pédagogique : Ce projet permet de mettre en pratique la construction d’un **système de menus** robuste (gestion d’état, anti-rebond, affichage dynamique), un savoir-faire transversal qui s’applique identiquement, que la sortie pilotée soit un écran LCD, OLED, une matrice de LED ou un anneau lumineux.

3 Composants Utilisés

3.1 Composants du cahier des charges (version complète)

lightgray Composant	Rôle dans le projet
LCD 16x2 I ² C	Affichage de texte défilant et du système de menus
OLED 0,96" (SSD1306)	Affichage graphique : icônes, formes, textes en mode bit-map
Matrice LED 8x8 MAX7219	Animations personnalisées par multiplexage lignes/colonnes
Afficheur 7 segments TM1637	Affichage d'un compteur numérique
Boutons (x4)	Navigation et validation dans le système de menus
Arduino Uno	Microcontrôleur assurant la logique de menu et le pilotage des afficheurs

TABLE 1 – Composants prévus dans le cahier des charges initial du projet.

3.2 Composants utilisés dans la simulation Tinkercad

lightgray Réf- erence	Quantité	Composant
U1	1	Arduino Uno R3
U2	1	Écran LCD 16x2 avec module I ² C basé sur le PCF8574 (adresse 0x20)
S1, S2	2	Boutons poussoirs (navigation « Suivant » / « Valider »)
RING1	1	Anneau NeoPixel de 12 LED RGB adressables

TABLE 2 – Composants effectivement câblés dans la simulation Tinkercad.

Remarque : La simulation remplace l'OLED, la matrice MAX7219 et l'afficheur TM1637 par un **anneau NeoPixel** (LED RGB adressables en série, protocole à un fil), ce qui permet de tester rapidement la logique de menu et de génération d'animations colorées avant l'intégration des écrans définitifs prévus dans le cahier des charges.

4 Notions Électroniques Clés

- **Protocoles I²C et SPI – différences, brochages, vitesses :** le LCD et l'OLED SSD1306 communiquent généralement en I²C (2 fils : SDA/SCL, vitesse modérée, adressage par adresse unique), alors que la matrice MAX7219 utilise le protocole **SPI** (3 à 4 fils : MOSI, SCK, CS, vitesse plus élevée, sans adressage mais avec sélection par broche *Chip Select*).
- **Multiplexage de lignes et colonnes (matrice LED) :** le contrôleur MAX7219 pilote une matrice 8x8 en allumant rapidement une ligne à la fois (balayage), donnant l'illusion d'un affichage stable grâce à la persistance rétinienne, ce qui réduit drastiquement le nombre de broches nécessaires (8+8 au lieu de 64).
- **Génération de caractères personnalisés (ROM/RAM du LCD) :** les écrans LCD à base de contrôleur HD44780 disposent d'une mémoire *CGRAM* permettant de définir

jusqu'à 8 caractères personnalisés (pictogrammes) en plus des caractères standards stockés en *CGROM*.

- **Gestion de l'anti-rebond (debouncing) sur boutons** : un bouton poussoir mécanique génère de multiples transitions parasites (rebonds) lors de son actionnement ; le code doit donc filtrer ces transitions, par exemple en ne déclenchant l'action qu'au front descendant (transition HIGH→LOW) suivi d'une courte temporisation, comme illustré dans le code de ce projet.

5 Schéma du Montage

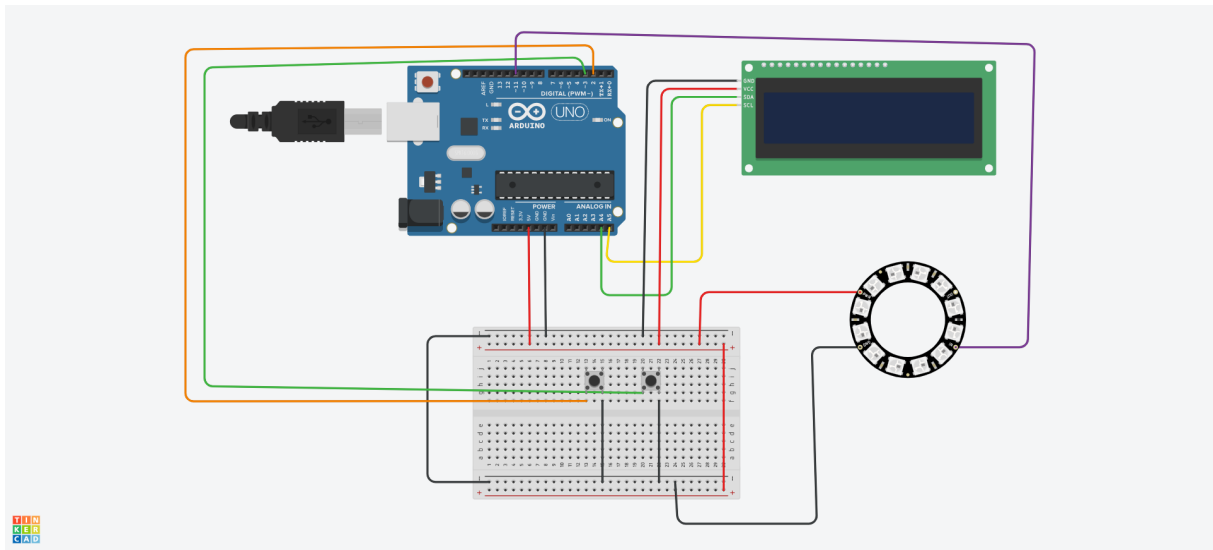


FIGURE 2 – Schéma de câblage du montage (Arduino Uno, LCD I²C, anneau NeoPixel, boutons poussoirs).

6 Code Arduino

Le programme ci-dessous implémente le système de menus interactifs : affichage du menu sur le LCD, navigation et validation via deux boutons avec anti-rebond, et exécution de trois animations distinctes sur l'anneau NeoPixel.

```

1 #include <LiquidCrystal_I2C.h>
2 #include <Adafruit_NeoPixel.h>
3
4 // --- Configuration du LCD (I2C) ---
5 LiquidCrystal_I2C lcd(0x20, 16, 2);
6
7 // --- Configuration de l'Anneau NeoPixel ---
8 #define PIN_NEOPIXEL 11
9 #define NUMPIXELS 12 // Change 16 ou 24 si tu prends un anneau plus grand
10 Adafruit_NeoPixel anneau(NUMPIXELS, PIN_NEOPIXEL, NEO_GRB + NEO_KHZ800);
11
12 // --- Configuration des Boutons ---
13 const int btnSuivant = 2;
14 const int btnValider = 3;
15
16 int etatMenu = 0; // 0=Chenillard, 1=Couleurs, 2=Effacer
17 const int MAX_MENU = 2;

```

```

18
19 // Variables pour l'anti-rebond
20 bool dernierEtatBtnS = HIGH;
21 bool dernierEtatBtnV = HIGH;
22
23 void setup() {
24     lcd.init();
25     lcd.backlight();
26     lcd.print("MENU ANIMATION");
27     delay(1500);
28
29     // Initialisation NeoPixel
30     anneau.begin();
31     anneau.setBrightness(50); // Luminosit (0 255)
32     anneau.show(); // teint tout par d faut
33
34     // Boutons en PULLUP interne (activ s sur tat BAS/LOW)
35     pinMode(btnSuivant, INPUT_PULLUP);
36     pinMode(btnValider, INPUT_PULLUP);
37
38     afficherMenu();
39 }
40
41 void loop() {
42     // --- Bouton "Suivant" ---
43     bool lectureS = digitalRead(btnSuivant);
44     if (lectureS == LOW && dernierEtatBtnS == HIGH) {
45         etatMenu++;
46         if (etatMenu > MAX_MENU) etatMenu = 0;
47         afficherMenu();
48         delay(200);
49     }
50     dernierEtatBtnS = lectureS;
51
52     // --- Bouton "Valider" ---
53     bool lectureV = digitalRead(btnValider);
54     if (lectureV == LOW && dernierEtatBtnV == HIGH) {
55         executerAction();
56         delay(200);
57     }
58     dernierEtatBtnV = lectureV;
59 }
60
61 // --- Fonctions d'affichage et d'action ---
62
63 void afficherMenu() {
64     lcd.clear();
65     lcd.setCursor(0, 0);
66     lcd.print("Action LED:");
67     lcd.setCursor(0, 1);
68     if (etatMenu == 0) lcd.print("> 1. Chenillard");
69     if (etatMenu == 1) lcd.print("> 2. Couleurs");
70     if (etatMenu == 2) lcd.print("> 3. Effacer");
71 }
72
73 void executerAction() {
74     anneau.clear();
75     anneau.show();
76
77     if (etatMenu == 0) {
78         // Animation : Tourne en bleu, une LED la fois
79         for(int i = 0; i < NUMPIXELS; i++) {
80             anneau.setPixelColor(i, anneau.Color(0, 150, 255)); // Bleu

```

```

81     anneau.show();
82     delay(100);
83     anneau.setPixelColor(i, anneau.Color(0, 0, 0)); // teint
84 }
85 anneau.show(); // Laisse l'anneau teint la fin
86 }
87 else if (etatMenu == 1) {
88     // Affichage : D grad fixe multi-couleurs
89     for(int i = 0; i < NUMPIXELS; i++) {
90         int rouge = (i * 255) / NUMPIXELS;
91         int bleu = 255 - rouge;
92         int vert = (i % 2 == 0) ? 100 : 0;
93         anneau.setPixelColor(i, anneau.Color(rouge, vert, bleu));
94     }
95     anneau.show();
96 }
97 else if (etatMenu == 2) {
98     // Effacer complétement
99     anneau.clear();
100    anneau.show();
101 }
102 }

```

Listing 1 – Code source Arduino du système de menus et d'animations (simulation Tinkercad).

7 Analyse et Discussion du Code

Le code peut être décomposé en cinq blocs fonctionnels principaux :

1. **Déclarations et initialisation (setup)** : l'écran LCD I²C (adresse 0x20) et l'anneau NeoPixel (12 LED, broche 11) sont initialisés. La luminosité de l'anneau est limitée à 50/255 via `setBrightness()` afin de ménager la consommation et le confort visuel. Les boutons sont configurés en `INPUT_PULLUP`, ce qui signifie qu'ils sont actifs à l'état LOW lorsqu'ils sont pressés (résistance de tirage interne activée).
2. **Boucle principale (loop) et anti-rebond** : à chaque itération, l'état courant des deux boutons est comparé à leur état précédent (`dernierEtatBtnS`, `dernierEtatBtnV`). Une action n'est déclenchée que lors d'un **front descendant** (`lecture == LOW && dernierEtat == HIGH`), c'est-à-dire au moment précis où le bouton passe de relâché à pressé. Un court `delay(200)` après chaque action limite les rebonds mécaniques résiduels – une solution simple bien qu'imparfaite, qui pourrait être remplacée par une temporisation basée sur `millis()` pour ne pas bloquer le programme.
3. **Navigation dans le menu (afficherMenu)** : la variable `etatMenu` (0 à 2) détermine l'option actuellement sélectionnée. Le bouton « Suivant » incrémente cette variable et la fait revenir à 0 après la dernière option (`MAX_MENU`), réalisant un **menu circulaire**. Le LCD est effacé puis réaffiché avec un curseur (>) indiquant l'option active.
4. **Exécution de l'action (executerAction)** : selon la valeur de `etatMenu`, l'anneau est d'abord remis à zéro (`anneau.clear()`), puis l'animation correspondante est lancée. Le **chenillard** (option 0) allume puis éteint successivement chaque LED en bleu avec un délai de 100 ms, créant un effet de rotation. Les **couleurs** (option 1) appliquent un dégradé calculé arithmétiquement à partir de l'indice `i` de chaque LED (interpolation rouge/bleu, alternance de vert). L'option **effacer** (option 2) éteint simplement l'anneau.
5. **Calcul du dégradé de couleurs** : les composantes rouge et bleue sont calculées par interpolation linéaire en fonction de la position `i` sur l'anneau (`rouge = (i * 255) / NUMPIXELS`, `bleu = 255 - rouge`), tandis que la composante verte alterne entre 100 et 0 selon la parité de l'indice, produisant un motif chromatique varié sans calcul complexe.

Points forts du code :

- Anti-rebond fonctionnel basé sur la détection de front, plutôt qu’une simple lecture d’état brut.
- Structure modulaire claire séparant affichage du menu (`afficherMenu`) et exécution des actions (`executerAction`).
- Utilisation pertinente de `NUMPIXELS` comme constante, facilitant l’adaptation à un anneau de taille différente (16 ou 24 LED).

Pistes d’amélioration possibles :

- Remplacer les `delay()` (200 ms après chaque action, 100 ms dans le chenillard) par une gestion non bloquante via `millis()`, afin que la lecture des boutons reste réactive même pendant une animation.
- Étendre la structure de menu existante pour piloter, en plus de l’anneau NeoPixel, les modules visés par le cahier des charges (OLED SSD1306 en I²C, matrice MAX7219 en SPI, afficheur TM1637), chacun pouvant correspondre à une nouvelle option de menu.
- Ajouter un troisième bouton « Retour » ou une temporisation d’inactivité pour revenir automatiquement au menu principal après l’exécution d’une animation.

8 Ressources Complémentaires

- **Vidéo** – *OLED Display SSD1306 Arduino : Complete Tutorial*
<https://www.youtube.com/watch?v=PrIAnDZ9dp8>
- **Vidéo** – *MAX7219 LED Matrix 8x8 Arduino animations*
<https://www.youtube.com/watch?v=FLRaQ5caBko>

9 Conclusion

Ce projet d’affichage avancé met en évidence la diversité des technologies d’affichage disponibles pour Arduino (I²C, SPI, LED adressables) et l’importance d’une architecture logicielle de menu robuste, indépendante du périphérique de sortie final. La simulation Tinkercad, bien qu’elle se limite à un écran LCD et un anneau NeoPixel, valide la logique de navigation, l’anti-rebond et la génération d’animations colorées – une base directement extensible vers l’intégration de l’écran OLED, de la matrice MAX7219 et de l’afficheur TM1637 prévus dans la version complète du projet.